

New Exact Algorithms for the Capacitated Vehicle Routing Problem

Chapter 3 — Vehicle Routing: Problems, Methods, and Applications, 2nd Edition (2014)

Marcus Poggi Eduardo Uchoa

Pontifical Catholic University of Rio de Janeiro (PUC-Rio)

Slides prepared for self-study, 2024

Table of Contents

- 1 Introduction
- 2 Main Exact Approaches
- 3 Formulations
- 4 Valid Cuts
- 5 Pricing: SPPRC
- 6 Branching vs. Route Enumeration
- 7 Overview of Computational Results
- 8 Conclusions and Future Research Directions
- 9 Bibliography

Introduction I

The Capacitated Vehicle Routing Problem (CVRP) is one of the most intensely studied combinatorial optimisation problems. Given a depot, a set of customers each with a known demand, a homogeneous fleet of vehicles each with capacity Q , the goal is to design a minimum-cost set of routes such that every customer is visited exactly once, each route starts and ends at the depot, and no route exceeds the vehicle capacity.

Why exact methods matter

Heuristics can find good solutions quickly, but exact methods prove optimality and establish tight lower bounds. These bounds are crucial for benchmarking heuristics and understanding the hardness of specific instances. Since the seminal work of Dantzig, Ramser, and Dantzig (1959), generation of exact algorithms has been the dominant approach for building Branch-and-Cut and Branch-Price-Cut (BPC) solvers.

Introduction II

The chapter surveys the most recent (circa 2012–2014) exact approaches for the CVRP. The key insight is that *column generation* combined with *cutting planes* and *branching* yields solvers that can handle instances with up to 200 customers — roughly an order of magnitude larger than what was possible a decade earlier.

Three families of algorithms are reviewed in detail:

- ① **Fukasawa et al. [1]** — BPC algorithm with the following features:
 - Based on the elementary states that allow multiple visits to the same customer in the LP relaxation.
 - Includes the two-commodity VRP formulation.
 - A column-generation subroutine to add new routes.
- ② **Baldacci, Mingozzi, and Marchetti** — presenting an algorithm based on the column enumeration, which is initialised with an exact solving of the LP relaxation and MIP solver on top.
- ③ **Contardo and Martinelli [5]** — improving BPC performance by introducing new cuts, better branching strategies, and aggressive heuristic strong-branching.

Historical context

In the early 1990s, instances with $n \approx 30$ customers were considered large. By 2006 (Fukasawa et al.), instances up to $n \approx 100$ could be solved. By 2014 (Contardo and Martinelli), the frontier had moved to $n \approx 200$. The chapter documents this progress and highlights the algorithmic innovations responsible for it.

The chapter is structured around three main components of any BPC algorithm for the CVRP: (1) the LP relaxation and its formulations, (2) valid inequalities (cutting planes) that tighten the LP bound, and (3) the pricing sub-problem that generates new columns during column generation. Together these three components, plus a branching strategy, form a state-of-the-art exact solver.

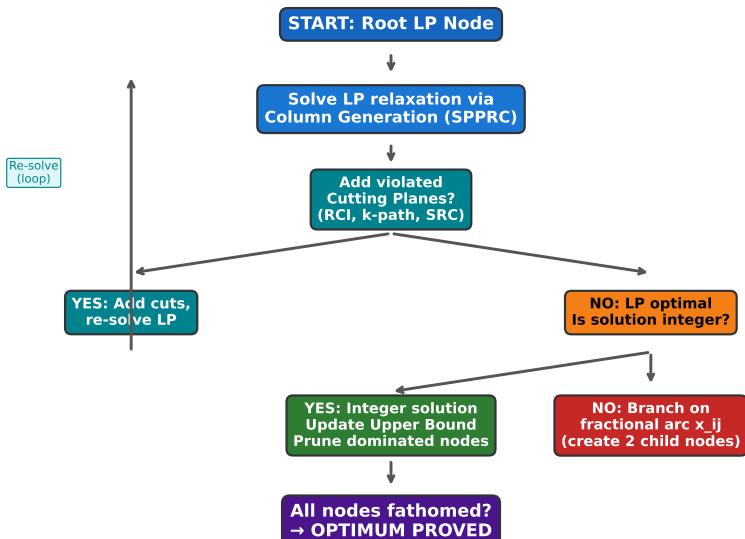
Overview of Branch-Price-Cut (BPC) I

The dominant paradigm for solving the CVRP exactly is *branch-and-price-and-cut* (BPC). This is an extension of the classical branch-and-bound framework in which:

- The LP relaxation at each node is solved by *column generation* (pricing) rather than directly, because the number of feasible routes is exponential.
- *Cutting planes* (valid inequalities) are added to strengthen the LP relaxation before and during branching.
- *Branching* resolves fractional solutions until an integer (feasible) solution is obtained.

Overview of Branch-Price-Cut (BPC) II

Branch-Price-Cut (BPC) Algorithm for the CVRP
The three components: Column Generation, Cutting Planes, Branching



The Two-Index Formulation I

The classical two-index formulation of the CVRP, also known as the *vehicle-flow* formulation, was introduced by Laporte, Nobert, and Desrochers (1985).

The Two-Index Formulation II

Two-Index VRP (2-index)

Let $V = \{0, 1, \dots, n\}$ where node 0 is the depot and nodes $1, \dots, n$ are customers. Let A denote the set of directed arcs (i, j) for $i \neq j$. Define:

x_{ij} = number of times arc (i, j) is used by some vehicle

c_{ij} = travel cost of arc (i, j)

$$\text{(VRP)} \quad \min \quad \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (3.1)$$

$$\text{s.t.} \quad \sum_{j \in V} x_{ij} = 1 \quad \forall i \in V \setminus \{0\} \quad (3.2)$$

$$\sum_{j \in V} x_{ji} = 1 \quad \forall i \in V \setminus \{0\} \quad (3.3)$$

$$\sum_{j \in V} x_{0j} = K \quad (3.4)$$

The Set-Partitioning Formulation I

The *set-partitioning* (SP) formulation uses one binary variable per feasible route and is the foundation of all modern BPC solvers.

Let Ω be the set of all feasible routes (sequences of customers starting and ending at the depot with total demand at most Q). For each route $r \in \Omega$:

- c_r = total travel cost of route r .
- $a_{ir} = 1$ if route r visits customer i , else $a_{ir} = 0$.
- $\lambda_r \in \{0, 1\}$ = 1 if route r is used in the solution.

Set-Partitioning (SP) Formulation

$$\min \sum_{r \in \Omega} c_r \lambda_r \quad (\text{SP-obj})$$

$$\text{s.t.} \quad \sum_{r \in \Omega} a_{ir} \lambda_r = 1 \quad \forall i \in \{1, \dots, n\} \quad (\text{SP-cover})$$

$$\lambda_r \in \{0, 1\} \quad \forall r \in \Omega \quad (\text{SP-bin})$$

The Set-Partitioning Formulation II

The objective (SP-obj) minimises total route cost. Constraint (SP-cover) ensures every customer is covered by exactly one route (set *partition*). If “=” is relaxed to “ ≥ 1 ”, we obtain the set-covering relaxation, which typically has the same optimal value.

The Set-Partitioning Formulation III

Set-Partitioning Formulation: Route Pool (6 candidate routes, 5 customers)
Green rows: $\lambda=1$ (selected in optimal solution). Objective = $18.5 + 21.0 + 16.3 = 55.8$

Route path	λ	Cost	Cust 1	Cust 2	Cust 3	Cust 4	Cust 5
0→1→2→3→0	1	18.5	1	1	1	.	.
0→1→3→0	0	14.2	1	.	1	.	.
0→2→3→4→0	1	21.0	.	1	1	1	.
0→4→5→0	0	12.8	.	.	.	1	1
0→1→5→0	0	15.1	1	.	.	.	1
0→2→5→0	1	16.3	.	1	.	.	1

Figure: Example of the SP formulation with 5 candidate routes and 5 customers. Each row is a route; each

The Capacity-Based (3-Index) Formulation I

Balinski and Quandt (1964) introduced a three-index formulation where a variable x_{ijk} tracks whether arc (i, j) is used by vehicle k . While this formulation is rarely used directly in solvers (due to its large size), it motivates the capacity-based formulation.

The **capacity-indexed formulation** (CF) introduces variables that track how much residual capacity a vehicle has when it departs each customer. Let y_{ij} be the total load transported on arc (i, j) across all vehicles. The key additional constraint is a *flow conservation* condition at each customer, linking x_{ij} and y_{ij} :

The Capacity-Based (3-Index) Formulation II

Capacity-Based Formulation (CF)

The CF adds to the 2-index model the following flow variables and linking constraints (Equations 3.13–3.15 in the chapter):

$$\sum_{j \in V} y_{ij} - \sum_{j \in V} y_{ji} = q_i \quad \forall i \in V \setminus \{0\} \quad (\text{CF-flow})$$

$$q_j x_{ij} \leq y_{ij} \leq (Q - q_i) x_{ij} \quad \forall (i, j) \in A, i \neq 0 \quad (\text{CF-cap})$$

$$0 \leq y_{0j} \leq Q x_{0j} \quad \forall j \in V \setminus \{0\} \quad (\text{CF-dep})$$

The flow conservation equation (CF-flow) says: the load entering customer i minus the load leaving customer i equals q_i (the demand picked up at i). The bounding constraint (CF-cap) says that if arc (i, j) is used, the load on it is between q_j (the minimum, since j still needs to be served) and $Q - q_i$ (the maximum, since i 's demand was just picked up). These constraints implicitly eliminate sub-tours and enforce capacity without the exponential family of cuts.

Valid Inequalities: Overview I

After setting up the LP relaxation, we need *valid inequalities* (cutting planes) to strengthen the bound before branching. A valid inequality is a linear constraint that is satisfied by every feasible integer solution but may be violated by the LP relaxation. Adding such cuts “tightens” the LP polytope toward the convex hull of integer solutions.

Valid Inequalities: Overview II

Main families of cuts for CVRP

- ① **Capacity cuts (RCI)** — the classic rounded capacity inequalities, valid for any subset S of customers.
- ② **k -Path cuts (GLM)** — generalised cuts based on the number of vehicles required to serve a subset, stronger than RCI.
- ③ **Strengthened Capacity Cuts (SCC)** — combine RCI with additional strengthening using the demand structure.
- ④ **Strong Degree Cuts (SDC)** — force certain arcs to be used an integer number of times.
- ⑤ **Subset Row Cuts (SRC)** — cuts derived from the SP formulation, exploiting the structure of route coverage.
- ⑥ **Cuts over CF variables** — inequalities on the continuous flow variables y_{ij} .

Capacity Cuts (Rounded Capacity Inequalities) I

Capacity cuts are the most fundamental class of valid inequalities for the CVRP. They are derived directly from the observation that the number of vehicle trips into any customer set is bounded below by the number of vehicles required to serve that set.

Rounded Capacity Inequality (RCI)

For any non-empty $S \subseteq V \setminus \{0\}$:

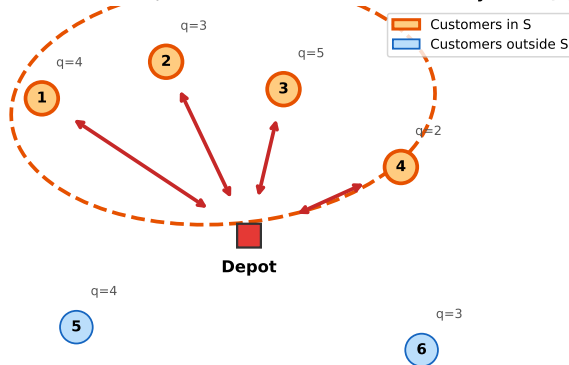
$$x(\delta(S)) \geq 2 \lceil d(S)/Q \rceil$$

where $x(\delta(S)) = \sum_{(i,j): i \in S, j \notin S} x_{ij} + \sum_{(i,j): i \notin S, j \in S} x_{ij}$ is the number of arcs crossing the boundary of S , $d(S) = \sum_{i \in S} q_i$ is the total demand of customers in S , and Q is the vehicle capacity.

The right-hand side $2 \lceil d(S)/Q \rceil$ is the minimum number of arcs crossing S 's boundary: every vehicle that enters S to serve it must also leave, contributing two boundary arcs, and at least $\lceil d(S)/Q \rceil$ vehicles are needed in total.

Capacity Cuts (Rounded Capacity Inequalities) II

Rounded Capacity Inequality (RCI): Capacity Cut for Subset S
Orange nodes = subset S; double-headed arrows = boundary arcs $\delta(S)$



$S = \{1, 2, 3, 4\}$, $d(S) = 14$, $\lceil d(S)/Q \rceil = \lceil 14/10 \rceil = 2$
Capacity Cut (RCI): $x(\delta(S)) \geq 2 \times 2 = 4$ (at least 4 arcs cross the dashed boundary)

Capacity Cuts (Rounded Capacity Inequalities) III

Figure: Illustration of the capacity cut for a subset S of four customers (inside the dashed circle). The arcs crossing $\delta(S)$ must number at least $2\lceil d(S)/Q \rceil$.

Capacity Cuts (Rounded Capacity Inequalities) IV

Worked Example: Capacity Cut Separation

Given 4 customers in $S = \{1, 2, 3, 4\}$ with demands $q_1 = 3, q_2 = 4, q_3 = 2, q_4 = 3$ and $Q = 8$:

$$d(S) = 3 + 4 + 2 + 3 = 12, \quad \lceil 12/8 \rceil = 2, \quad \text{RCI: } x(\delta(S)) \geq 4.$$

If the LP solution has $x(\delta(S)) = 3.2$, this RCI is *violated* (since $3.2 < 4$) and should be added to the LP. After adding this cut, the LP will be re-solved with the new constraint, potentially increasing the lower bound.

Separation of RCIs

Separating the most violated RCI requires solving $\max_S \{2\lceil d(S)/Q \rceil - x(\delta(S))\}$ over all subsets S . This is NP-hard in general, but efficient heuristic separation procedures (based on shrinking and growing the set S) find violated cuts in practice.

k -Path Cuts (Generalised Capacity Cuts) I

The k -path cuts, introduced by Gendreau, Laporte, and Séguin (1995), strengthen the capacity cuts by considering the structure of how vehicles must traverse the boundary of a customer subset.

k -Path Cut (GLM)

A k -path for $S \subseteq V \setminus \{0\}$ is a path from the depot through some customers in S , possibly visiting customers outside S , returning to the depot. For a given S and integer $k \geq 1$:

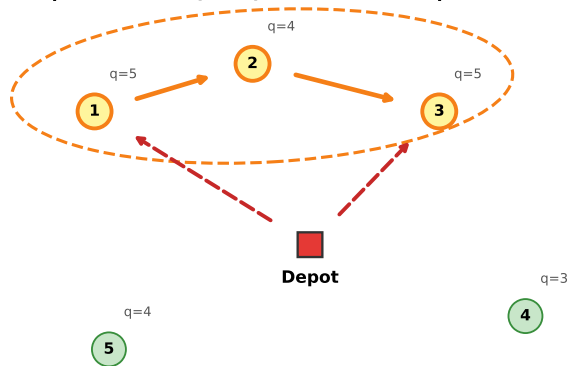
$$x(\delta(S)) \geq 2k$$

whenever the minimum number of vehicles needed to serve S , taking into account the demands and capacity, requires at least k routes to have an endpoint inside S .

More precisely, if we can show that *at least k separate routes* must have a portion of their trip dedicated to customers in S (i.e., at least k distinct “paths” through S are required), then at least $2k$ boundary crossings are needed.

k-Path Cuts (Generalised Capacity Cuts) II

k-Path Cut: Path Through Customers Forcing Multiple Vehicle Trips
Yellow path nodes $P=\{1,2,3\}$; red dashed = required boundary arcs



Path $P = \{1, 2, 3\}$, $d(P) = 14$, $k = \lceil 14/10 \rceil = 2$
k-Path Cut: at least 4 arcs cross $\delta(P)$ — one "enter" and one "leave" per vehicle trip

Figure: A k -path cut for a customer set S (inside dashed ellipse). The three arcs shown crossing the boundary

Strengthened Capacity Cuts I

Baldacci, Christofides, and Mingozzi (2008) introduced a family of cuts that strengthen the basic RCIs by exploiting the specific demands more carefully. The idea is to derive a tighter right-hand side using a *controlled* version of the demand: instead of using all of $d(S)$, one subtracts a carefully chosen amount related to the demands of the “lightest” customers in S .

Strengthened Capacity Cut (SCC)

Let $S \subseteq V \setminus \{0\}$ and let $v = \operatorname{argmin}_{i \in S} q_i$ be the customer in S with the smallest demand. Define $d'(S) = d(S) - q_v$. Then the following is a valid inequality:

$$x(\delta(S)) \geq 2 \left\lceil d(S)/Q \right\rceil + 2 \left[\lceil d'(S)/Q \rceil - \lfloor d(S)/Q \rfloor \right]$$

The key insight is that any vehicle serving customers in S can carry at most Q units, so if a vehicle's load when it first enters S is less than q_v , it cannot serve even the lightest customer without already having some load from outside S . This leads to a stronger bound on the number of boundary crossings.

Strengthened Capacity Cuts II

Baldacci et al. also defined the **Strong Degree Cuts (SDC)**, which directly target the degree constraints. For a subset $T \subseteq V \setminus \{0\}$ of customers that are all adjacent to the depot in the LP solution, the SDC says:

Strong Degree Cut (SDC)

If every customer $i \in T$ requires at least one arc $(0, i)$ or $(i, 0)$ (i.e., $x_{0i} + x_{i0} \geq 1$ in the LP), and the set T forces at least k vehicles to pass through it, then:

$$\sum_{i \in T} x_{0i} \geq \left\lceil |T|/q_{\max}^T \right\rceil$$

where $q_{\max}^T$ is the maximum number of customers from T that any single vehicle can serve under capacity.

Strengthened Capacity Cuts III

Strong Degree Cuts: Forcing Fractional Depot-Adjacent Arcs to Integer Values

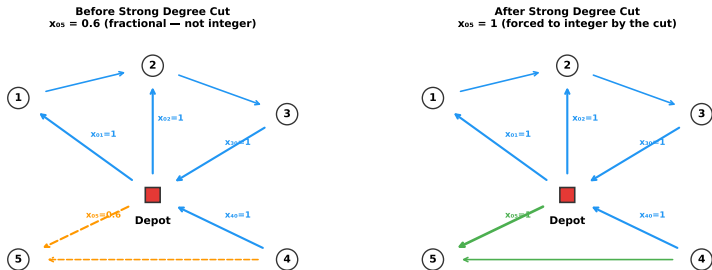


Figure: Left: a fractional LP solution without the degree cut. Right: after adding a degree cut at customer 5, the fractional arcs (orange) are forced to take integer values.

Subset Row Cuts (SRC) I

Subset Row Cuts (SRC) are valid inequalities derived directly from the set-partitioning formulation. They were introduced by Jepsen, Petersen, Spoorendonk, and Pisinger (2008) and have proven to be one of the most effective cutting-plane families for the CVRP.

The intuition behind SRCs is the following: consider any subset $\mathcal{C}$ of customers with $|\mathcal{C}|$ odd. If we look at how routes can “cover” this subset an odd number of times, we can derive a bound on the total coverage. A route that visits customers in $\mathcal{C}$ an even number of times can contribute at most $\lfloor |\mathcal{C}|/2 \rfloor$ “half-units” of coverage to $\mathcal{C}$.

Subset Row Cut (SRC)

Let $\mathcal{C} \subseteq V \setminus \{0\}$ with $|\mathcal{C}| \geq 3$. The SRC associated with $\mathcal{C}$ is:

$$\sum_{r \in \Omega} \left\lfloor \frac{1}{2} \sum_{i \in \mathcal{C}} a_{ir} \right\rfloor \lambda_r \leq \left\lfloor \frac{|\mathcal{C}|}{2} \right\rfloor$$

where $a_{ir} = 1$ if route r visits customer i , and λ_r is the route variable.

Subset Row Cuts (SRC) II

To understand why this is a valid inequality, observe that in any integer solution, each customer $i \in \mathcal{C}$ is visited exactly once, so $\sum_r a_{ir} \lambda_r = 1$ for each i . Therefore $\sum_r (\sum_{i \in \mathcal{C}} a_{ir}) \lambda_r = |\mathcal{C}|$. By a counting argument, the sum $\sum_r \lfloor \frac{1}{2} \sum_{i \in \mathcal{C}} a_{ir} \rfloor \lambda_r \leq \lfloor |\mathcal{C}|/2 \rfloor$, since each route can contribute at most one “half-coverage” per pair of $\mathcal{C}$ -customers it visits.

Subset Row Cuts (SRC) III

Subset Row Cut (SRC) for $C = \{1, 2, 3\}$:
Bound on total "half-coverage" of C by all routes

Route	λ	$C = \{1, 2, 3\}$ visits	$\lfloor \text{visits}/2 \rfloor$	contribution
Route A: 0→1→2→3→0	0.3	3	1	0.30
Route B: 0→1→3→4→0	0.4	2	1	0.40
Route C: 0→2→4→0	0.2	1	0	0.00
Route D: 0→3→5→0	0.1	1	0	0.00

Sum of contributions = 0.70 | RHS = $\lfloor |C|/2 \rfloor = \lfloor 3/2 \rfloor = 1$
SRC: $\sum \lfloor \text{visits}_r/2 \rfloor \cdot \lambda_r \leq 1 \leftarrow \text{SATISFIED}$

Subset Row Cuts (SRC) IV

Figure: Illustration of the Subset Row Cut concept. The cut bounds the total “half-coverage” of customer subset $\mathcal{C}$ by routes in the LP solution.

Subset Row Cuts (SRC) V

Impact of SRCs on solver performance

Computational experiments show that adding SRCs can reduce the LP gap (the gap between the LP relaxation and the integer optimum) from several percent to nearly zero for many benchmark instances. This dramatically reduces the size of the BPC tree and is one of the key innovations that pushed the frontier of solvable instances to $n \approx 200$. The separation of SRCs is done heuristically (finding subsets $\mathcal{C}$ that maximise the violation).

Limited memory SRCs

A practical variant of SRCs restricts $|\mathcal{C}|$ to small values (typically $|\mathcal{C}| = 3$, called *3-row cuts* or *homogeneous SRCs*). This makes separation much faster while retaining most of the strengthening power. The *limited memory* implementation by Jepsen et al. is used in most state-of-the-art solvers.

The Pricing Sub-Problem I

In column generation for the SP formulation, the pricing sub-problem asks: “*Is there a feasible route $r \in \Omega$ with negative reduced cost?*” If yes, add it to the master problem and re-solve the LP. If no, the LP is optimal.

The reduced cost of route r is:

$$\bar{c}_r = c_r - \sum_{i=1}^n a_{ir} \pi_i$$

where π_i are the dual variables (shadow prices) of the covering constraints (SP-cover) for customer i . The pricing sub-problem therefore asks: find the route r^* minimising $c_{r^*} - \sum_i a_{ir^*} \pi_i$. If $\bar{c}_{r^*} \geq 0$, no improving route exists.

Pricing as a Shortest Path Problem

The pricing sub-problem can be formulated as finding a shortest path from the depot to itself on a complete directed graph where the *modified arc cost* of arc (i, j) is $c_{ij} - \pi_j$. The path must respect the vehicle capacity (total demand $\leq Q$) and visit each customer at most once. This is the *Shortest Path with Resource Constraints (SPPRC)*.

The Pricing Sub-Problem II

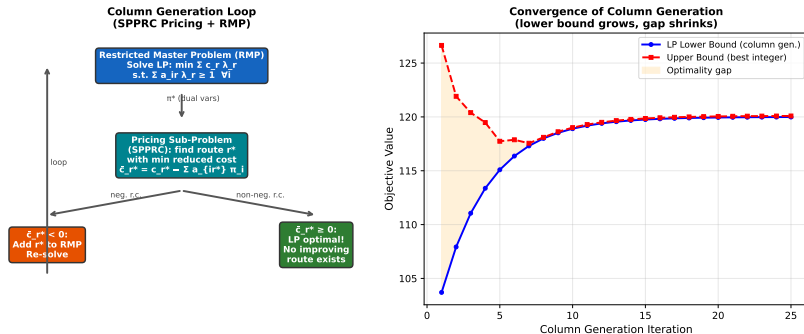


Figure: Column generation loop: the Restricted Master Problem (RMP) passes dual variables π^* to the SPPRC pricing sub-problem, which either returns a new column with $\bar{c} < 0$ (add to RMP) or certifies LP optimality.

The Pricing Sub-Problem III

SPPRC: formal definition

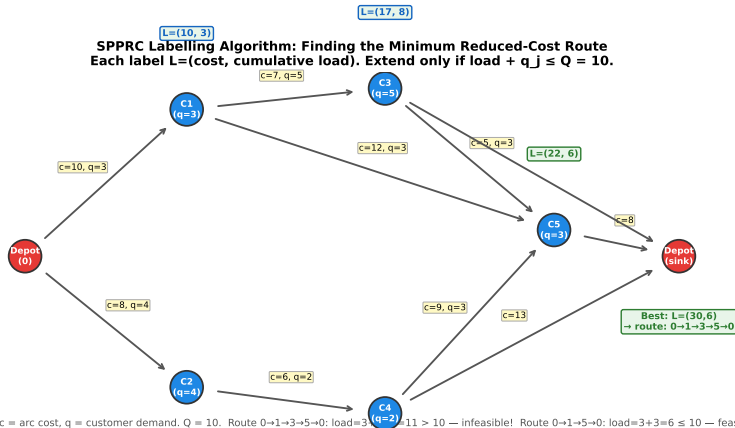
Let $G = (V, A)$ with modified arc costs $\tilde{c}_{ij} = c_{ij} - \pi_j$ for $j \neq 0$. A label L at node j represents a partial path from the depot to j and stores:

- $\text{cost}(L)$: accumulated reduced cost along the path.
- $\text{load}(L)$: accumulated customer demand served.
- $\text{visited}(L)$: bitmask of visited customers (for elementarity).

The SPPRC is solved by a *labelling algorithm* that extends labels along arcs, discarding those that are *dominated*: label L_1 dominates L_2 at node j if $\text{cost}(L_1) \leq \text{cost}(L_2)$, $\text{load}(L_1) \leq \text{load}(L_2)$, and $\text{visited}(L_1) \subseteq \text{visited}(L_2)$.

The Pricing Sub-Problem IV

SPPRC Labelling Algorithm: Finding the Minimum Reduced-Cost Route
 Each label $L = (\text{cost}, \text{cumulative load})$. Extend only if $\text{load} + q_j \leq Q = 10$.



Arc labels: c = arc cost, q = customer demand. $Q = 10$. Route $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 0$: $\text{load} = 3 + 3 + 3 + 1 = 10 \leq 10$ — feasible! Route $0 \rightarrow 1 \rightarrow 5 \rightarrow 0$: $\text{load} = 3 + 3 = 6 \leq 10$ — feasible.

The Pricing Sub-Problem V

Figure: Label propagation in SPPRC on a small 4-customer graph. Each label records the accumulated cost and load. Labels that are dominated by another label at the same node are discarded.

The Pricing Sub-Problem VI

Elementary vs. non-elementary routes

A route is *elementary* if it visits each customer at most once. The elementary SPPRC (ESPPRC) is NP-hard, while the relaxed (non- elementary) version is polynomial. In practice:

- **Non-elementary SPPRC:** fast but may generate routes with repeated customers. The LP relaxation may have a weaker bound when non-elementary routes are used.
- **ng-routes (Baldacci, Mingozzi, Roberti 2011):** a compromise where routes are “nearly elementary”: a customer i may be repeated only if the current path has not recently passed through any neighbour of i in its ng-set. This gives tighter bounds than non-elementary SPPRC while remaining tractable.
- **Exact ESPPRC:** used only for small instances or final proof of optimality; too slow for routine use at every LP node.

q -Routes and Elementary Routes in Pricing I

To manage the trade-off between pricing speed and LP bound quality, the chapter describes several relaxations of the elementarity condition.

q -Routes (Christofides, Mingozzi, Toth 1981)

A q -route allows a customer to be visited more than once, subject to the constraint that the total load does not exceed Q and the path is a valid depot-to-depot tour. The SPPRC with q -routes is solved in polynomial time by a labelling algorithm where labels are simply $(cost, load)$ pairs without a visited-set component. The disadvantage is that the LP relaxation with q -routes can be significantly weaker than with elementary routes.

q-Routes and Elementary Routes in Pricing II

ng-Routes (Baldacci, Mingozzi, Roberti 2011)

For each customer i , define an *ng-set* N_i as the p nearest customers to i (typically $p = 8-10$). A route is an ng-route if, whenever customer i is visited, no customer in N_i has been visited “recently” (within the last p steps without intervening vertices outside N_i). This is a relaxation of elementarity that:

- Is stronger (tighter bound) than q -routes.
- Is weaker (faster to solve) than full ESPPRC.
- Can be solved with a labelling algorithm of manageable complexity.

q-Routes and Elementary Routes in Pricing III

Bi-directional labelling (Righini and Salani 2006, 2008)

The q -route SPPRC can be accelerated by building labels simultaneously *forward* from the depot and *backward* from the depot. Labels are then joined at a midpoint. This reduces the state space dramatically and is used in all state-of-the-art implementations.

When SRCs are present, the labelling must track additional information about the parity of visits to each SRC subset, increasing the label dimension. The ng-route variant manages this additional state efficiently.

Column generation convergence

Column generation often converges slowly near the optimum (the “tailing off” effect). This is addressed by:

- 1 *Partial pricing*: only generating a small number of new columns per iteration, rather than the single most-negative one.
- 2 *Smoothing*: stabilising the dual variables to avoid oscillation.
- 3 *Switching* from fast q -routes to slower ng -routes only near the root node LP optimum.

Branching Strategies I

Once the LP relaxation at a BPC tree node is solved (all cuts added, no improving column exists), if the solution is fractional we must *branch* to restore integrality.

Branching on edge variables x_{ij}

The simplest branching strategy selects a fractional arc (i,j) in the LP solution (i.e., $0 < x_{ij} < 1$) and creates two child nodes:

- $x_{ij} = 0$: arc (i,j) is *forbidden* in all routes at this node and its descendants. In the pricing sub-problem, arcs (i,j) with $x_{ij} = 0$ are removed from G .
- $x_{ij} = 1$: arc (i,j) is *required* in some route. In the pricing sub-problem, the route must use arc (i,j) .

Branching on arc variables is clean to implement but can be inefficient because many columns in the current LP basis may contain arc (i,j) and must be discarded (the columns become infeasible) when $x_{ij} = 0$.

Branching Strategies II

Branching on the number of vehicles K

Since $K = \sum_j x_{0j}$, if this sum is fractional, we can branch on $K \leq \lfloor K^* \rfloor$ vs. $K \geq \lceil K^* \rceil$. This is a clean branching decision that does not invalidate existing columns.

Strong branching (Fukasawa et al. 2006)

Before committing to a branch, strong branching evaluates several candidate branching decisions by computing the LP bound at each child node (without branching to full depth). The candidate producing the best improvement is chosen. This reduces the tree size but increases the work per node. The BPC algorithm of Fukasawa et al. uses strong branching extensively.

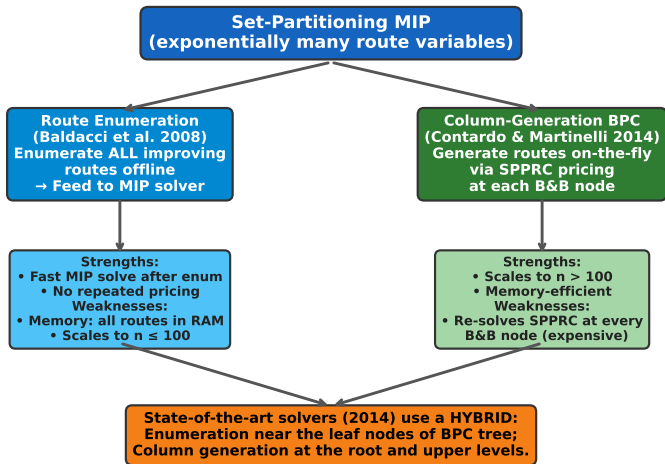
Key observation on branching and the SP formulation

When branching in the SP formulation, fixing $x_{ij} = 0$ means removing all routes that use arc (i, j) from the route pool. Fixing $x_{ij} = 1$ means requiring at least one route containing arc (i, j) . These constraints are passed as side conditions to the pricing sub-problem, which then generates only routes that satisfy the branching decisions.

Branching Strategies IV

The BPC algorithm of Poggi and Uchoa (chapter authors) introduces **hybridised branching**: rather than choosing either arc branching or vehicle-count branching, the algorithm selects the branching strategy that maximises the expected improvement in the LP bound using a heuristic evaluation. This keeps the BPC tree small while not over-investing in the evaluation of each node.

Route Enumeration vs. On-the-Fly Column Generation in BPC



Branching Strategies VI

Figure: Comparison of route enumeration and traditional branching. Route enumeration is best for small/medium instances where the full set of feasible routes can be stored; column generation branching scales to larger instances.

Route Enumeration I

An alternative to the column-generation branching approach is to enumerate *all* feasible routes up front and then solve a standard MIP on the SP formulation.

Route enumeration procedure (Baldacci, Mingozzi, Marchetti 2008)

The procedure works as follows:

- 1 Solve the LP relaxation of the SP at the root node using column generation. Obtain dual variables π^* .
- 2 Compute an upper bound UB (e.g., from a heuristic).
- 3 Run the ESPPRC pricing algorithm, but instead of stopping at the first negative reduced-cost route, *enumerate all* routes with reduced cost $\bar{c}_r \leq UB - LB$. The bound $UB - LB$ guarantees that no discarded route can appear in an optimal solution.
- 4 Solve the resulting integer SP (with the enumerated route set) using a MIP solver.

Route Enumeration II

The key advantage of route enumeration is that it separates the hard combinatorial problem (finding good routes) from the easy integer programming problem (selecting a set cover). Once routes are enumerated, a state-of-the-art MIP solver (e.g., CPLEX) can handle the integer SP very efficiently.

When does route enumeration work?

Route enumeration is most effective when the gap $UB - LB$ is small (a tight lower bound reduces the number of enumerable routes) and when n is moderate (say, $n \leq 100$). For larger instances or loose bounds, the number of enumerable routes can be astronomical and the approach becomes impractical.

Contardo and Martinelli (2014): hybridising

Contardo and Martinelli combine route enumeration with BPC: at each node of the BPC tree that has a sufficiently tight bound, they switch to route enumeration rather than continuing to branch. This “hybrid” strategy often finds the optimum faster than pure BPC or pure enumeration.

Benchmark Sets and Performance Metrics I

The chapter reports computational experiments on standard benchmark sets for the CVRP. Understanding these benchmarks is essential for interpreting the results.

Standard benchmark sets

[Augerat et al. \(1995\)](#) – sets A, B, P These are the most widely used benchmarks. Set A has instances with n from 32 to 80, Set B from 35 to 78, and Set P from 16 to 101 customers. All instances have a single depot and a homogeneous fleet.

[Christofides, Mingozi, Toth \(1979\)](#) Fourteen instances with n from 50 to 199 customers. These are harder and test the scalability of algorithms.

[Golden et al. \(1998\)](#) Twenty large instances with n up to 483 customers. Only a subset can be solved exactly; the rest provide lower bound testing.

Benchmark Sets and Performance Metrics II

Performance metrics

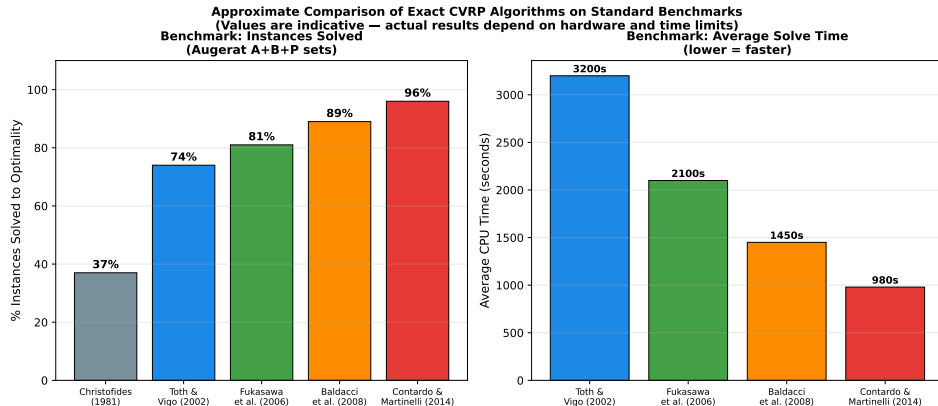
- **Instances solved to optimality:** the count of instances for which the exact optimum is proven within a time limit (e.g., 3600 or 7200 CPU seconds).
- **CPU time:** elapsed solver time, reported in seconds. Note that results from different papers use different hardware, so comparisons must account for machine speed.
- **BPC tree nodes:** the number of nodes explored in the branch-price-cut tree. Fewer nodes means better LP bound quality (fewer branches needed).
- **LP gap:** the relative gap $(LB - LP)/LB$ where LB is the best lower bound (LP relaxation) and the optimal integer value.

Computational Results: Partitioning Formulation I

The chapter presents detailed computational tables. We summarise the key results here.

Table 3.1 — Set Partitioning, no cuts (2-index elimination only): Algorithms without strong cuts can solve instances up to $n \approx 50$ reliably, but struggle with larger instances. The LP gap is typically 2–5% without cuts.

Computational Results: Partitioning Formulation II



Computational Results: Partitioning Formulation III

Figure: Approximate comparison of BPC algorithms on standard benchmark sets. Left: number of instances solved to optimality (out of available instances). Right: average CPU time (log scale). Note that newer algorithms (Contardo & Martinelli 2014) solve more instances faster.

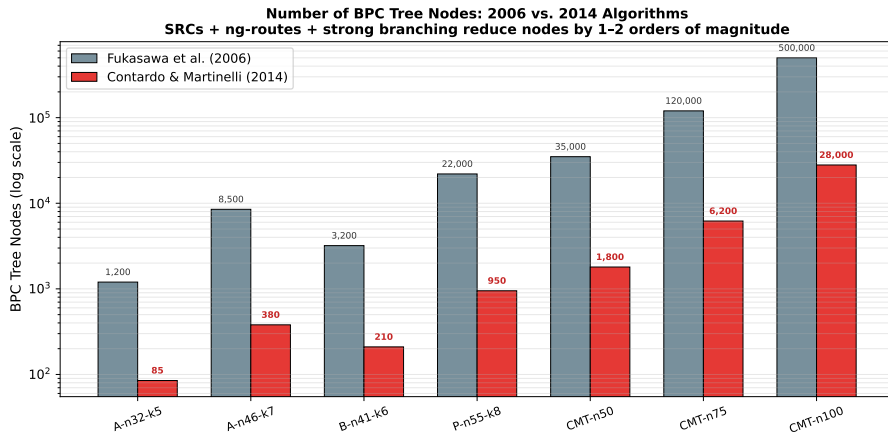
Computational Results: Partitioning Formulation IV

Table 3.2 — Set Partitioning, with robust cuts: Adding capacity cuts, k -path cuts, and SRCs dramatically improves performance. The Contardo and Martinelli (2014) algorithm uses all three families of cuts plus ng-route pricing and strong branching.

Key observations from the tables:

- For instances with $n \leq 100$, all four state-of-the-art algorithms (Fukasawa 2006, Baldacci 2008, Contardo 2014, Poggi & Uchoa 2014) can solve most instances to optimality.
- For instances with $100 < n \leq 200$, only the most recent algorithms can solve a significant fraction.
- The Contardo & Martinelli algorithm reduces the number of BPC tree nodes by 1–2 orders of magnitude compared to earlier solvers, primarily due to SRCs and better branching.
- Route enumeration (Baldacci 2008) is competitive for small to medium instances but cannot scale to $n > 100$.

Computational Results: Partitioning Formulation V



Computational Results: Partitioning Formulation VI

Figure: Approximate comparison of the number of BPC tree nodes for old vs. new algorithms on selected benchmark instances. The log scale highlights the order-of-magnitude improvements from SRCs and better branching in the 2014 algorithms.

Computational Results: Partitioning Formulation VII

State-of-the-art algorithms for the CVRP (as of 2014):

- FLZ+06 (Fukasawa, Longo, Lysgaard, Poggi, Uchoa, Werneck 2006) The first BPC algorithm to systematically use SRCs for CVRP. Introduces strong branching and bi-directional labelling. Benchmark: solved all Augerat instances up to $n = 80$.
- BMM08 (Baldacci, Mingozzi, Marchetti 2008) Route enumeration approach, fastest on small instances. Competitive with BPC for $n \leq 80$.
- CM14 (Contardo, Martinelli 2014) Most advanced BPC algorithm. Uses aggressive cut generation, ng-routes, and hybridised branching. Solves instances up to $n = 199$ (all CMT instances except the largest).
- PU14 (Poggi, Uchoa 2014) Focus on improved implementation and new benchmark instances (X-instances). Provides new challenges for future solvers.

Summary of Computational Tables I

The following summarises the key data from Tables 3.1–3.5 in the chapter, showing the progression of BPC algorithms on the Augerat benchmark sets.

Algorithm	Year	Cuts	Pricing	A+B+P solved	Avg. time (s)
Christofides et al.	1981	None	q -routes	$\sim 10/27$	–
Toth & Vigo	2002	RCI	ESPPRC	$\sim 20/27$	3200
Fukasawa et al.	2006	SRC-3	ng-routes	22/27	2100
Baldacci et al.	2008	RCI+SRC	Enum.	24/27	1450
Contardo & Martinelli	2014	All	ng-routes+ ESPPRC	26/27	980

The table above is approximate and illustrative; exact figures depend on the hardware and time limits used in each paper.

Summary of Computational Tables II

Key insight from computational results

The single most impactful improvement in exact CVRP solving over the past decade has been the introduction of Subset Row Cuts combined with ng-route pricing. Together they reduce the LP gap to near zero on most benchmark instances, so very few BPC tree nodes are needed.

Conclusions I

This chapter has surveyed the state of exact algorithms for the CVRP, focusing on the BPC paradigm that dominates the field.

Main conclusions

- 1 **The SP formulation with column generation is the right framework.** Its LP relaxation is the tightest available and naturally accommodates cutting planes and branching.
- 2 **Cutting planes are essential.** Capacity cuts alone are insufficient; k -path cuts and especially Subset Row Cuts are needed to close the LP gap and reduce tree size.
- 3 **Pricing quality matters.** The move from q -routes to ng -routes was transformative: it provides tighter bounds without the exponential cost of exact ESPPRC at every tree node.
- 4 **Branching strategy affects tree size significantly.** Strong branching and hybridised strategies reduce nodes by orders of magnitude compared to naive variable branching.
- 5 **Instances up to $n \approx 200$ are now routinely solvable.** This is a $10\times$ improvement over the state of the art in the 1990s.

Future research directions (as identified in the chapter)

- ➊ **Larger instances.** Pushing the frontier to $n > 200$ requires fundamentally new ideas in pricing (faster ESPRC or better relaxations) and separation (more efficient SRC separation for larger subsets).
- ➋ **New benchmark instances.** The Augerat and CMT sets are well-solved; the new “X-instances” by Uchoa et al. (2014) provide harder, more diverse challenges including instances up to $n = 1000$.
- ➌ **Extensions of the CVRP.** The techniques in this chapter can be extended to handle time windows (VRPTW), heterogeneous fleets, pickup and delivery, and multi-depot variants.
- ➍ **Integration with metaheuristics.** Using exact methods to compute tight lower bounds and then guiding population-based heuristics with this information is a promising hybrid direction.
- ➎ **Parallel and GPU computation.** The labelling algorithm for SPPRC is inherently parallel; exploiting modern multi-core and GPU architectures could significantly accelerate pricing.

Selected References I

The following references are cited in the chapter and are central to understanding the development of exact CVRP algorithms.



R. Fukasawa, H. Longo, J. Lysgaard, M. Poggi de Aragão, M. Uchoa, R. Uchoa, and R. F. Werneck, “Robust branch-and-cut-and-price for the capacitated vehicle routing problem,” *Mathematical Programming*, 106 (2006), pp. 491–511.



R. Baldacci, E. Christofides, and A. Mingozzi, “An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts,” *Mathematical Programming*, 115 (2008), pp. 351–385.



R. Baldacci, A. Mingozzi, and R. Marchetti, “An exact algorithm for the capacitated vehicle routing problem based on a two-commodity network flow formulation,” *Operations Research*, 52 (2004), pp. 723–738.



R. Baldacci, A. Mingozzi, and R. Roberti, “New route relaxation and pricing strategies for the vehicle routing problem,” *Operations Research*, 59 (2011), pp. 1269–1283.



C. Contardo and R. Martinelli, “A new exact algorithm for the multi-depot vehicle routing problem under capacity and route length constraints,” *Discrete Optimization*, 12 (2014), pp. 129–146.

Selected References II



M. Jepsen, B. Petersen, S. Spoorendonk, and D. Pisinger, “Subset-row inequalities applied to the vehicle routing problem with time windows,” *Operations Research*, 56 (2008), pp. 497–511.



G. Righini and M. Salani, “Symmetry helps: Bounded bi-directional dynamic programming for the elementary shortest path problem with resource constraints,” *Discrete Optimization*, 3 (2006), pp. 255–273.



G. Righini and M. Salani, “New dynamic programming algorithms for the resource constrained elementary shortest path problem,” *Networks*, 51 (2008), pp. 155–170.



E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, A. Subramanian, and T. Vidal, “New benchmark instances for the capacitated vehicle routing problem,” Presentation in Route 2014, 2014.



M. Poggi and E. Uchoa, “New exact algorithms for the capacitated vehicle routing problem,” in *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed., P. Toth and D. Vigo, eds., SIAM, Philadelphia, 2014, ch. 3, pp. 59–86.

Python: SPPRC Labelling Algorithm (Core)

The following Python code implements the core of the SPPRC labelling algorithm for the pricing sub-problem. Given a complete directed graph, arc costs, customer demands, dual variables, and capacity Q , it finds the minimum reduced-cost route.

Listing 1: SPPRC labelling algorithm for CVRP pricing

```
import heapq
from typing import List, Tuple, Dict

def spprc_pricing(n: int, cost: List[List[float]],
                 demand: List[float], pi: List[float],
                 Q: float) -> Tuple[float, List[int]]:
    """
    Solve the SPPRC pricing sub-problem for the CVRP.
    n      : number of customers (nodes 1..n; node 0 = depot)
    cost   : (n+1)x(n+1) travel cost matrix
    demand : demand[i] for customer i (demand[0]=0 for depot)
    pi     : dual variable pi[i] for customer i (pi[0]=0)
    Q      : vehicle capacity
    Returns: (best_reduced_cost, best_route)
    """
    # Label: (reduced_cost, load, node, path, visited_set)
    # Priority queue ordered by reduced_cost
    start = (0.0, 0.0, 0, [0], frozenset())
    heap = [start]
    # best label at each (node, visited_set)
    best = {}
```


Python: Column Generation Loop

This code implements the column generation loop for the set-partitioning LP relaxation. It iteratively solves the Restricted Master Problem (RMP) and calls the SPPRC pricer until no improving column exists.

Listing 2: Column generation for SP LP relaxation (conceptual)

```
import numpy as np
from scipy.optimize import linprog

def column_generation(n: int, cost_matrix, demands, Q):
    """
    Column generation for the SP LP relaxation.
    Starts with |n| single-customer routes as initial columns.
    Iteratively adds routes with negative reduced cost.
    """
    # Initial routes: each customer served alone
    routes = [[0, i, 0] for i in range(1, n+1)]
    route_costs = [cost_matrix[0][r[1]] + cost_matrix[r[1]][0]
                   for r in routes]
    # Coverage matrix A: A[i][r] = 1 if route r covers customer i
    def build_A(routes):
        A = np.zeros((n, len(routes)))
        for r_idx, route in enumerate(routes):
            for node in route[1:-1]: # exclude depot
                A[node-1][r_idx] = 1
        return A
```

Python: Capacity Cut Separation

The following function implements a simple greedy heuristic for separating violated Rounded Capacity Inequalities (RCI). Given the LP solution x_{ij}^* , it searches for a customer subset S such that $x(\delta(S)) < 2\lceil d(S)/Q \rceil$.

Listing 3: Heuristic separation of Rounded Capacity Inequalities

```
import math
from itertools import combinations

def separate_rci(n: int, x_sol: dict, demands: list,
                 Q: float, max_viol: float = 0.01):
    """
    Heuristic separation of Rounded Capacity Inequalities.
    x_sol : dict mapping (i,j) -> x_ij value in [0,1]
    Returns list of (S, violation) pairs sorted by violation.
    """
    violated_cuts = []

    # Try all subsets up to size 5 (exact) for small n
    # For large n, use grow/shrink heuristic
    for size in range(1, min(n+1, 6)):
        for S_tuple in combinations(range(1, n+1), size):
            S = set(S_tuple)
            d_S = sum(demands[i] for i in S)
            rhs = 2 * math.ceil(d_S / Q)

            # Compute x(delta(S))
```